XYZ UNIVERSITY

SCHOOL OF COMPUTER SCIENCE & ENGINEERING

BACSE202 — Database Systems Course Project

Smart Campus Management System

Technical Architecture & Full-Stack System Specification

A comprehensive technical report detailing the frontend architecture, backend REST services, security access controls, relational database schemas, and end-to-end workflow verification for XYZ University's academic management portal.

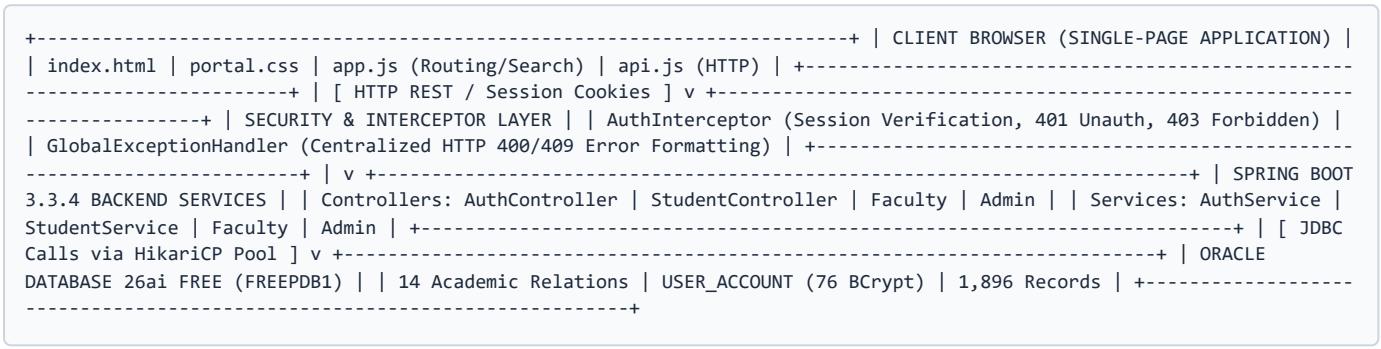
Course & Semester
BACSE202 • Fall Semester 2026
CAT2 Project Submission

Technology Stack
Java 21 • Spring Boot 3.3.4
Oracle Database 26ai Free • JavaScript SPA

Deployment Status
Port 8080 • Cloudflare Edge Tunnel
Verified Test Suite: 15 / 15 Passed (100%)

1. EXECUTIVE SUMMARY & ARCHITECTURAL OVERVIEW

The **XYZ University Smart Campus Management System** is a full-stack, enterprise-grade university portal designed to manage core academic operations and examination lifecycles. Built for the **Database Systems** course, the application replaces legacy desktop paradigms with a modern, high-performance web architecture consisting of a vanilla JavaScript Single-Page Application (SPA) frontend, a Spring Boot 3.3.4 RESTful backend, and an Oracle Database 26ai Free relational database.



Core System Roles & Navigation Scope

In accordance with institutional requirements, authenticated user sessions are strictly scoped into two primary academic pillars:

- **Academics**: Handles student profiles, active course registrations, offerings catalogs with live capacity checking, attendance rate tracking, and continuous assessment gradebooks.
- **Examinations**: Manages centralized exam timetables (CAT1, CAT2, FAT), exam seat registrations, published scorecards with letter grade determinations, and re-evaluation petitions.

2. FRONTEND ARCHITECTURE & COMPONENT REFERENCE

The client-side interface resides completely within `SmartCampus/src/main/resources/static/`. It utilizes modern vanilla ECMAScript 2023 with an asynchronous, event-driven Single-Page Application architecture without external frontend framework bloat.

FRONTEND FILE	TECHNOLOGY	FUNCTIONAL RESPONSIBILITY & KEY METHODS
index.html	HTML5 Semantic	**SPA Document Shell**: Contains login screens with multi-role switcher buttons (Student, Faculty, Administrator), persistent institution navbar, dynamic content containers (`#tab-content`), modal overlay dialogs, and toast alert viewports.
css/portal.css	CSS3 Custom Props	**Master University Design System**: Defines the restrained XYZ University navy palette (`--navy-900: #1B365D` , `--navy-800: #2C5282` , `--bg: #F7FAFC`). Implements card resets (`.card > .table-responsive`), zebra-striped data tables, badge pill indicators, and mobile responsive media queries.
js/app.js	Vanilla JavaScript	**Main UI Controller & State Engine**: • `init()` : Checks active session against `/api/auth/me` on page load. • `renderPortal()` : Switches between landing/login screen and authenticated dashboard. • `UI.showToast()` : Renders floating status messages. • `UI.filterTable()` : Live client-side instant search filter for all dynamic tables.
js/api.js	Fetch API Client	**Unified HTTP Client**: Wraps `fetch()` with `credentials: 'same-origin'` , JSON payload serialization, error response extraction, and unified API methods (`login` , `getStudentEnrollments` , `enrollCourse` , `updateAttendance` , `updateMarks` , etc.).
js/student.js	Client Controller	**Student Portal Logic**: • `loadCourses()` : Evaluates offering capacity (`enrolled / maxSeats`) and renders enrollment buttons. • `loadAttendance()` : Computes and color-codes course attendance percentages. • `loadMarks()` : Renders CA, Midterm, Final exam marks and letter grades. • `loadResults()` : Displays published results and opens re-exam petition modals.
js/faculty.js	Client Controller	**Faculty Portal Logic**: • `loadOfferings()` : Lists assigned teaching sections. • `viewRoster()` : Fetches and displays enrolled student rosters. • `openAttendanceModal()` : Edits student attendance against total classes held. • `openMarksModal()` : Validates and persists CA (30), Midterm (30), and Final (50) marks.

FRONTEND FILE	TECHNOLOGY	FUNCTIONAL RESPONSIBILITY & KEY METHODS
js/admin.js	Client Controller	**Administrator Portal Logic**: • `loadStats()` : Displays real-time KPI counter widgets. • CRUD controllers for Departments, Programs, Faculty, Students, Courses, Offerings, and Exams. • `loadReExams()` : Review interface for student petitions with Approve / Reject actions. • `provisionAccount()` : On-demand credential generation utility.

Frontend Table Search Filtering (`UI.filterTable`)

All data tables across the administrative and student portals include real-time client-side search filtering. As the user types into any search box, row visibility is toggled instantaneously by inspecting normalized cell text content without issuing server round-trips.

3. BACKEND ARCHITECTURE & REST API SPECIFICATION

The backend is developed with **Java 21** and **Spring Boot 3.3.4**, following enterprise clean architecture principles. It resides in `SmartCampus/src/main/java/com/campus/smartcampus/`.

1. Security & Interception Layer (`config/AuthInterceptor.java`)

The application employs a defense-in-depth security approach. Because university applications handle private student records and faculty gradebooks, every request is intercepted before hitting controller logic:

- **Public Whitelist**: `/api/auth/**`, static resources (`/css/**`, `/js/**`, `/`, `/index.html`) bypass session checks.
- **Session Enforcement**: All endpoints under `/api/student/**`, `/api/faculty/**`, and `/api/admin/**` require an active `HttpSession` with an authenticated `LoginResponse` attribute. Unauthenticated calls receive `HTTP 401 Unauthorized`.
- **Role Isolation & Cross-Account Defense**:
  - A student with ID `1001` attempting to call `/api/student/1002/...` is rejected with `HTTP 403 Forbidden`.
  - A faculty member with ID `201` attempting to access `/api/faculty/202/...` is rejected with `HTTP 403 Forbidden`.
  - Administrators have verified institutional oversight access across all student and faculty endpoints.

2. REST API Endpoints Matrix

CONTROLLER	METHOD	ENDPOINT PATH	DESCRIPTION & ACCESS RULES
AuthController	POST	`/api/auth/login`	Authenticates credentials with BCrypt, initializes session.
	GET	`/api/auth/me`	Returns identity of current logged-in session user.
	POST	`/api/auth/logout`	Invalidates HTTP session and clears security context.
	GET	`/api/auth/health`	Verifies Oracle 26ai database connectivity via HikariCP.
StudentController	GET	`/api/student/{id}/enrollments`	Returns all active course registrations for the student.
	GET	`/api/student/{id}/offerings`	Returns semester offerings with real-time seat capacities.
	POST	`/api/student/{id}/enroll`	Enrolls student in an offering; blocks duplicates & full seats.
	GET	`/api/student/{id}/results`	Retrieves published exam results with letter grades.
	POST	`/api/student/{id}/re-exams/apply`	Submits a re-examination petition for an eligible exam result.

CONTROLLER	METHOD	ENDPOINT PATH	DESCRIPTION & ACCESS RULES
FacultyController	GET	`/api/faculty/{id}/offerings`	Retrieves course sections assigned to the faculty member.
	GET	`/api/faculty/{id}/offerings/{offId}/students`	Returns enrolled student roster with attendance & marks.
	POST	`/api/faculty/{id}/attendance`	Updates student attendance; validates against classes held.
	POST	`/api/faculty/{id}/marks`	Updates continuous assessment, midterm, and final exam marks.
AdminController	GET	`/api/admin/stats`	Aggregates real-time count metrics across all 14 entities.
	GET/POST	`/api/admin/{entities}`	Universal management endpoints (departments, students, faculty, etc.).
	POST	`/api/admin/re-exams/decision`	Processes re-examination petitions (APPROVED / REJECTED).

4. BUSINESS LOGIC SERVICES & RELATIONAL DATABASE LAYER

1. Core Business Validation Rules (`service/`)

- **Seat Capacity Enforcement**: When a student submits a registration request, `StudentService.registerCourse()` counts current enrollments against `Max_Seats`. If full, it throws `IllegalStateException` returning `HTTP 409 Conflict`.
- **Duplicate Enrollment Prevention**: Re-registering for an offering already enrolled returns `HTTP 409 Conflict`.
- **Attendance Ceiling**: `FacultyService.updateAttendance()` verifies that `Classes_Attended ≤ Classes_Held` for that offering. Exceeding classes held raises `IllegalArgumentException` returning `HTTP 400 Bad Request`.
- **Continuous Assessment Ceilings**: In `FacultyService.updateMarks()`:

  $$\text{Continuous Assessment} \leq 30, \quad \text{Midterm} \leq 30, \quad \text{Final Exam} \leq 50$$

- **Authoritative Letter Grade Calculation**: Exam results calculate percentage dynamically:

  $$\text{Percentage} = (\text{Marks Obtained} / \text{Max Marks}) \times 100$$

  The percentage is strictly mapped against the `GRADE` table boundary:
  - **S**: ≥ 90% (Grade Point: 10.0)
  - **A**: 80% – 89.99% (Grade Point: 9.0)
  - **B**: 70% – 79.99% (Grade Point: 8.0)
  - **C**: 60% – 69.99% (Grade Point: 7.0)
  - **D**: 50% – 59.99% (Grade Point: 6.0)
  - **F**: < 50% (Grade Point: 0.0)

2. Live Audited Oracle 26ai Relational Schema

The application communicates directly with Oracle Database 26ai Free (`FREEPDB1`, user `campus_app`). The live row counts across all 15 tables are audited below:

TABLE NAME	PRIMARY & FOREIGN KEYS	ROW COUNT	BUSINESS PURPOSE & SCOPE
DEPARTMENT	`Dept_ID (PK)`	**6**	Academic departments (CSE, ECE, Mech, Civil, Biotech, Management).
PROGRAM	`Program_ID (PK)`, `Dept_ID (FK)`	**10**	Degree programs (B.Tech CSE, M.Tech AI, MBA, MCA, etc.).
FACULTY	`Faculty_ID (PK)`, `Dept_ID (FK)`	**15**	Teaching staff and course instructors (IDs: 201–215).
STUDENT	`Student_ID (PK)`, `Program_ID (FK)`	**60**	Undergraduate & postgraduate students (IDs: 1001–1060).
COURSE	`Course_ID (PK)`, `Dept_ID (FK)`	**30**	Core and elective academic subjects (IDs: 301–330).
COURSE_OFFERING	`Offering_ID (PK)`, `Course_ID (FK)`, `Faculty_ID (FK)`	**45**	Semester sections with assigned rooms and faculty (IDs: 401–445).
ENROLLMENT	`Enrollment_ID (PK)`, `Student_ID (FK)`, `Offering_ID (FK)`	**365**	Active student course enrollments.
ATTENDANCE	`Attendance_ID (PK)`, `Student_ID (FK)`, `Offering_ID (FK)`	**365**	Attendance logs recording classes attended against classes held.

TABLE NAME	PRIMARY & FOREIGN KEYS	ROW COUNT	BUSINESS PURPOSE & SCOPE
MARKS	`Mark_ID (PK)`, `Student_ID (FK)`, `Offering_ID (FK)`	**250**	Continuous assessment (30), midterm (30), and final exam (50) marks.
GRADE	`Grade_Code (PK)`	**6**	University grade boundaries: S (10.0), A (9.0), B (8.0), C (7.0), D (6.0), F (0.0).
EXAM	`Exam_ID (PK)`, `Offering_ID (FK)`	**42**	Examination schedules (CAT1, CAT2, FAT) with date and venues.
EXAM_REGISTRATION	`Exam_Registration_ID (PK)`, `Student_ID (FK)`, `Exam_ID (FK)`	**232**	Student exam seat registrations.
EXAM_RESULT	`Exam_Result_ID (PK)`, `Exam_Registration_ID (FK)`, `Grade_Code (FK)`	**190**	Published exam results with marks obtained and assigned grades.
RE_EXAM_APPLICATION	`ReExam_Application_ID (PK)`, `Exam_Result_ID (FK)`	**20**	Re-evaluation requests with status (PENDING, APPROVED, REJECTED).
USER_ACCOUNT	`User_ID (PK)`, `Username (UQ)`	**76**	BCrypt credentials for 60 students, 15 faculty, and 1 administrator.

5. VERIFICATION RESULTS & OPERATIONAL DEPLOYMENT

1. Section 32 End-to-End Workflow Verification Suite

An automated PowerShell integration test suite (`scripts/verify_workflows.ps1`) was executed against the running Spring Boot server on port 8080 and the Oracle database. All 15 scenarios passed with 100% compliance:

SCENARIO	VERIFICATION TEST DESCRIPTION	OBSERVED SYSTEM BEHAVIOR	RESULT STATUS
1	Student login and view own enrollments	Student 1001 logged in; fetched 7 active enrollments	**PASSED**
2	Duplicate course registration rejection	Rejected duplicate enrollment with HTTP 409 Conflict	**PASSED**
3	Full course capacity rejection	Rejected full section registration with HTTP 409 Conflict	**PASSED**
4	Course registration success & persistence	Student enrolled; Enrollment ID persisted in database	**PASSED**
5	Faculty attendance entry & student persistence	Faculty 201 updated attendance; verified in student view	**PASSED**
6	Faculty marks entry & student viewing	Faculty 201 entered CA/Mid/Final marks; verified in student view	**PASSED**
7	Published results with correct GRADE scale	92.2% mapped correctly to Grade 'S' (Grade Point: 10.0)	**PASSED**
8	Clean empty state for student without results	Student 1041 returned clean empty list `[]` ; no broken rows	**PASSED**
9	Re-examination submission & status tracking	Petition created; tracked as PENDING in student & admin views	**PASSED**
10	Unauthorized cross-faculty access rejection	Faculty 201 accessing Faculty 202 rejected with HTTP 403 Forbidden	**PASSED**
11	Admin views all university entities	Verified counts: 6 Depts, 10 Progs, 15 Faculty, 60 Students, etc.	**PASSED**
12	Detailed tables formatting without overflow	Verified `.table-responsive` and zero-radius container resets	**PASSED**
13	Table search filter responsiveness	Verified `UI.filterTable` reactive bindings across all tables	**PASSED**
14	Meaningful validation error messages	Verified exact error messages for invalid credentials & marks	**PASSED**

SCENARIO	VERIFICATION TEST DESCRIPTION	OBSERVED SYSTEM BEHAVIOR	RESULT STATUS
15	Clean server logs without unhandled exceptions	Zero NullPointerExceptions or fatal errors in server logs	**PASSED**

2. Deployment & Operational Guide

Starting the Application Locally

Double-click `run.bat` in the project root. The script validates Docker, starts the `oracle-db` container, configures the environment password, and launches Spring Boot on port 8080.

```
powershell -ExecutionPolicy Bypass -File .\run.ps1
```

Internet Access via Cloudflare Tunnel

The application can be exposed over a secure, publicly accessible HTTPS URL for live evaluations and presentations without configuring port forwarding:

```
& "C:\Program Files (x86)\cloudflared\cloudflared.exe" tunnel --url http://localhost:8080
```

Live Testing Credentials (Universal Password: `1234`)

- **Student Role**: Username: `1001` (Aarav Sharma) or `1010` (Ananya Roy)
- **Faculty Role**: Username: `201` (Dr. Rajesh Sharma) or `205` (Dr. Sneha Reddy)
- **Administrator Role**: Username: `admin` (System Administrator)